

SME FINANCIAL INTELLIGENCE PLATFORM

UNIFIED PRODUCT REQUIREMENTS DOCUMENT

12-Hour Hackathon Build | Team of 3 | Wema Bank Hackaholics

Single source of truth

This document freezes product scope, technical stack, API contracts, data structures, ML methodology, testing gates, deployment rules, ownership, and change-control. All teammates and AI agents must build against this document and the version-controlled repository contracts.

Version: 1.0 | Status: BUILD-LOCKED | Intended use: shared by all teammates and coding agents

0. TEAM ALIGNMENT RULES - READ FIRST	4
1. PRODUCT SUMMARY	4
2. PROBLEM STATEMENT	4
3. GOALS AND NON-GOALS	5
3.1 GOALS FOR THE HACKATHON	5
3.2 NON-GOALS	5
4. USERS AND PRIMARY USER JOURNEYS.....	5
4.1 CANONICAL DEMO JOURNEY	5
5. FUNCTIONAL REQUIREMENTS.....	6
5.1 PORTFOLIO DASHBOARD - P0	6
5.2 SME FINANCIAL HEALTH - P0.....	6
5.3 ACCOUNT AGGREGATION / OPEN BANKING SIMULATION - P0	6
5.4 CASH-FLOW FORECAST - P0.....	6
5.5 CREDIT READINESS - P0.....	6
5.6 LOAN WHAT-IF SIMULATOR - P0 / HEADLINE FEATURE.....	6
5.7 P1 / P2 FEATURES ONLY IF TIME REMAINS	7
6. CANONICAL ARCHITECTURE	7
7. TECHNOLOGY STACK - LOCKED	7
8. DATA ARCHITECTURE	8
8.1 CANONICAL ENTITIES.....	8
8.2 TRANSACTION CONVENTIONS - MUST NOT DIFFER ACROSS DEVICES	8
9. SYNTHETIC DATA SPECIFICATION	8
10. FINANCIAL INTELLIGENCE CALCULATIONS	9
10.1 CORE DAILY METRICS.....	9
10.2 PROTOTYPE FINANCIAL HEALTH SCORE	9
10.3 CREDIT-READINESS SCORE - LOCKED.....	9
11. FORECASTING SPECIFICATION	9
11.1 RECOMMENDED MODEL STACK	10
11.2 FEATURES	10
11.3 EVALUATION	10
12. LOAN WHAT-IF SIMULATION.....	10
13. API CONTRACT - FREEZE AND SHARE	10
13.1 API RESPONSE RULES	11

14. REPOSITORY STRUCTURE - SHARED SOURCE OF TRUTH	11
15. THREE-PERSON OWNERSHIP	12
15.1 PARALLEL-WORK RULE	12
16. TESTING STRATEGY AND ACCEPTANCE GATES	12
16.1 MINIMUM AUTOMATED TESTS	12
17. DEPLOYMENT - VERCEL + RENDER.....	13
17.1 DEPLOYMENT TOPOLOGY.....	13
17.2 REQUIRED ENVIRONMENT VARIABLES	13
17.3 DEPLOYMENT ACCEPTANCE	13
18. PRIVACY, SECURITY AND RESPONSIBLE AI.....	13
19. 12-HOUR BUILD SCHEDULE	13
20. CUT LIST IF BEHIND SCHEDULE	14
21. AI AGENT CONTRACT	14
21.1 MANDATORY AGENT INSTRUCTIONS	14
21.2 MASTER OPENCODE PROMPT	14
22. CHANGE CONTROL AND CONFLICT RESOLUTION.....	15
22.1 CHANGE REQUEST TEMPLATE.....	15
23. FINAL DEFINITION OF DONE	16
24. SUBMISSION CHECKLIST	16
25. CROSS-DEVICE TEAM SYNCHRONIZATION PACK.....	17
25.1 SOURCE-OF-TRUTH HIERARCHY	17
25.2 MANDATORY SHARED ARTIFACTS	17
25.3 REQUIREMENTS-GATHERING CONTRACT	17
25.4 EXACT PROCESS LOG FORMAT.....	18
25.5 SHARED SCHEMAS TO FREEZE	18
25.6 CONTRACT-CHANGE RULE	19
25.7 MASTER OPENCODE PROMPT — EXACT TEAM VERSION	19
26. RECOMMENDED ADDITIONAL CONTRACT FILES.....	21

0. Team Alignment Rules - Read First

Rule 1 - One repository wins

All devices must pull from the same GitHub repository. The repository, not an individual laptop, is the source of truth.

Rule 2 - Contracts before parallel coding

Freeze the API contract, database schema, naming conventions, environment variables, and demo dataset IDs before the team splits into parallel work.

Rule 3 - P0 before P1

No optional feature is started until the complete P0 journey works end-to-end on a shared environment.

Rule 4 - AI agents cannot redesign the product

Agents may implement assigned tasks but may not independently change the stack, endpoint shapes, data model, score weights, or user journey without a logged change decision.

Rule 5 - Synthetic demo only

The hackathon prototype uses synthetic data. It must not imply live Wema, live Open Banking, or production credit decisioning.

1. Product Summary

The SME Financial Intelligence Platform is a bank-side decision-support layer that turns SME financial activity into an understandable, forward-looking picture of the business. It analyses transactions and related authorised financial data, produces financial-health insights, forecasts 30/60/90-day liquidity, calculates an explainable prototype credit-readiness score, and simulates the effect of a proposed loan on future cash flow.

Core proposition: “The bank already sees the transactions. The platform helps the bank see the business.”

Product dimension	Decision
Primary customer	Commercial bank / SME banking team
Primary beneficiaries	SME customers, credit teams, relationship managers, portfolio/risk teams
Primary use case	Better-informed SME lending and monitoring
Prototype input	Synthetic Wema-like transaction data + synthetic consented external accounts
Primary outputs	Health, cash-flow intelligence, 30/60/90 forecast, credit readiness, loan simulation
Integration stance	Decision-support layer; does not replace core banking or official underwriting
Build window	12 hours

2. Problem Statement

Current problem: A bank can hold rich transaction data on an SME, yet raw transactions do not automatically become a coherent, forward-looking picture of business health. SMEs may also operate across multiple financial institutions. This can make a single-bank view incomplete when a customer is seeking credit or when a relationship manager wants to understand the business trajectory.

The platform targets the gap between transaction data and actionable financial decisions.

- Understand what the SME business is doing now.
- Estimate what its cash position may look like over the next 30, 60 and 90 days.
- Explain financial strengths and risks in plain language.
- Assess credit readiness using transparent prototype indicators.
- Run a proposed loan against the forecast to show potential liquidity pressure.
- Use customer-authorised external financial data when available, without assuming unrestricted access to unrelated third-party accounts.

3. Goals and Non-Goals

3.1 Goals for the hackathon

- Demonstrate a complete decision-support workflow from transaction data to lending insight.
- Use a clean React + TypeScript frontend and typed FastAPI backend with PostgreSQL.
- Create a realistic synthetic SME portfolio with different business behaviours.
- Produce an explainable forecast and credit-readiness score.
- Show the value of combining Wema-native data with consented external-account data through a mock Open Banking adapter.
- Deploy a working public prototype to Vercel + Render.
- Provide tests and clear handoff documentation so parallel work on different devices remains consistent.

3.2 Non-goals

- No live Wema core-banking integration.
- No live external-bank/Open Banking connection unless an authorised sandbox is actually provided.
- No automated loan approval or decline.
- No production underwriting policy claims.
- No LSTM/Transformer research project; simple explainable models are preferred.
- No accounting/ERP/inventory replacement.
- No multi-service/microservices architecture.
- No large customer-facing mobile app in the 12-hour scope.

4. Users and Primary User Journeys

User	Job to be done	MVP screen
SME relationship manager	Understand an SME quickly and identify financing opportunities/risks	Portfolio + SME detail
Credit analyst	Evaluate cash-flow capacity and loan impact	Forecast + Credit Readiness + Loan Simulator
Portfolio/risk manager	See which SMEs are healthy or under pressure	Portfolio dashboard
SME customer - future	Connect external accounts and understand financial position	Consent/customer view - optional

4.1 Canonical demo journey

1. Open the portfolio and choose “Ade’s Fashion Store”.
2. Show financial health, revenue trend, cash-flow stability, and recent inflows/outflows.
3. Show Wema account plus synthetic externally authorised accounts and the consent status.

4. Open 30/60/90 forecast and explain the key drivers.
5. Open Credit Readiness and show the score breakdown.
6. Enter a proposed loan, e.g. NGN 2,000,000 over 12 months.
7. Simulate the loan and compare projected liquidity before and after repayment.
8. Close with the value to the bank: better information before a lending decision, using data the bank is already authorised to use plus consented external data.

5. Functional Requirements

5.1 Portfolio Dashboard - P0

- List 5-10 synthetic SMEs.
- Show SME name, sector, financial-health score, forecast status, and credit-readiness score.
- Show portfolio counts: Healthy, Watch, High Pressure.
- Clicking an SME opens its detail page.

5.2 SME Financial Health - P0

- Calculate a 0-100 health score using deterministic indicators.
- Display average monthly inflow/outflow and net cash-flow trend.
- Display cash-flow stability and revenue consistency.
- Display business trend: growing, stable, declining, or volatile.
- Display top expense categories and recurring payment patterns.
- Provide 2-4 plain-language insights grounded in computed metrics.

5.3 Account Aggregation / Open Banking Simulation - P0

- Show the Wema account and 2-3 synthetic external business accounts.
- Show institution, masked account ID, account type, source, consent status, consent timestamp and expiry.
- Show which metrics are based on Wema-only data vs unified authorised data.
- Provide a mock “Connect account” state transition.

5.4 Cash-Flow Forecast - P0

- Show historical cash-flow series.
- Forecast 30/60/90 days.
- Show projected balance/liquidity and forecast status.
- Show key drivers: inflow trend, recurring obligations, seasonality and volatility.
- Label forecasts as estimates; never guarantee future balances.

5.5 Credit Readiness - P0

- Produce an explainable 0-100 prototype score.
- Show component scores and weights.
- Show positive and negative drivers.
- Explicitly label the score as prototype decision support, not Wema policy.

5.6 Loan What-If Simulator - P0 / headline feature

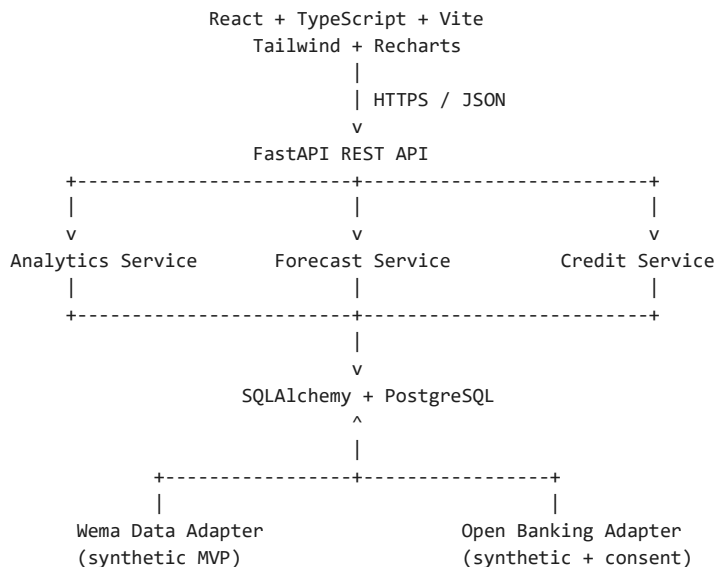
- Accept proposed loan amount, tenor, and monthly repayment estimate.
- Recalculate projected liquidity under the proposed repayment schedule.
- Compare before vs after at 30/60/90 days.

- Classify the simulated impact: manageable, watch, high pressure.
- Use decision-support wording rather than approve/decline wording.

5.7 P1 / P2 features only if time remains

- Customer-facing consent management.
- Downloadable SME decision brief.
- Deterioration alert timeline.
- LLM-generated narrative summary using only computed metrics.
- Live compliant Open Banking sandbox adapter.
- Authentication / RBAC beyond a simple demo role switch.

6. Canonical Architecture



ML / analytics: pandas + NumPy + scikit-learn
 Optional baseline: statsmodels ExponentialSmoothing
 Deployment: Vercel (frontend) + Render (FastAPI + PostgreSQL)

Architecture principle

The backend owns all business calculations. The frontend renders API results; it must not independently reimplement forecast, score, health, or simulation formulas. This prevents the three devices from producing different answers.

7. Technology Stack - Locked

Layer	Technology	Locked rule
Frontend	React + TypeScript + Vite	No alternate frontend framework for the main app
UI	Tailwind CSS	Use consistent spacing and component tokens
Charts	Recharts	One charting library only for MVP
Client data	TanStack Query + fetch/Axios	API results come from backend
Backend	Python + FastAPI	All server business logic here
Validation	Pydantic v2	Typed request/response schemas
ORM	SQLAlchemy 2.x	No direct SQL scattered through API routes

Migrations	Alembic	All schema changes are migrations
Database	PostgreSQL	Same database engine locally and on Render
Analytics/ML	pandas + NumPy + scikit-learn	No heavyweight ML framework in MVP
Optional forecasting baseline	statsmodels	Only if useful; not required
Backend tests	pytest + pytest-asyncio + httpx	Required for P0 services/endpoints
Frontend tests	Vitest + React Testing Library	Required for key components
E2E	Playwright	Optional if time remains
Version control	Git + GitHub	Mandatory single source of truth
Frontend deploy	Vercel	Production demo frontend
Backend deploy	Render Web Service	Production demo API
Database deploy	Render PostgreSQL	Production demo DB

8. Data Architecture

8.1 Canonical entities

Entity	Required fields	Owner
SME	id, business_name, business_type, location, business_age_months	Backend schema owner
Account	id, sme_id, institution_name, account_type, masked_account, source, is_primary, consent_status, consent_started_at, consent_expires_at	Backend schema owner
Transaction	id, account_id, date, amount, direction, category, channel, merchant_name, balance_after, reference	Backend schema owner
Loan	id, sme_id, principal, outstanding, monthly_repayment, tenor_months, repayment_status	Backend schema owner
ForecastPoint	sme_id, date, projected_net_cashflow, projected_balance, lower_bound, upper_bound optional	ML service owner
CreditReadiness	sme_id, total_score, component scores, reasons, generated_at	ML/service owner

8.2 Transaction conventions - MUST NOT differ across devices

- Amounts are numeric and represented in Nigerian Naira for the hackathon demo.
- Direction uses exactly CREDIT or DEBIT.
- Date/time is stored in UTC in the database and converted to Africa/Lagos only at the UI edge.
- Account IDs shown to users are masked. Synthetic raw IDs can remain internal to the fixture dataset.
- Transaction category is one controlled enum: sales, supplier, payroll, rent, utilities, transfer, loan_repayment, cash_withdrawal, other.
- All balance calculations must be derived consistently from transaction order; no route or frontend may invent balances.

9. Synthetic Data Specification

The synthetic dataset is not decoration. It is part of the product contract because it must produce a predictable, judge-friendly story on every device.

Fixture	Behaviour	Expected story
---------	-----------	----------------

ADE_FASHION_001	Growing revenue, seasonal spikes, stable supplier cycle	Healthy; strong credit readiness; positive forecast
TOLA_PHARMACY_002	Stable recurring inflows and outflows	Very stable; strong repayment capacity
LAGOS_BISTRO_003	High frequency, high volatility, weekend spikes	Watch; volatile forecast
KANO_ELECTRONICS_004	Large but irregular payments	Moderate; needs careful loan sizing
MART_DECLINE_005	Declining inflows, rising expenses, weaker liquidity	High pressure; lower readiness

Determinism rule

Use fixed fixture IDs and a fixed random seed so all three developers see the same results.

10. Financial Intelligence Calculations

10.1 Core daily metrics

```
daily_inflow = sum(CREDIT transactions for day)
daily_outflow = sum(DEBIT transactions for day)
net_cashflow = daily_inflow - daily_outflow
closing_balance = prior_balance + net_cashflow
```

10.2 Prototype financial health score

Health is separate from credit readiness. Use a simple 0-100 score and expose its components. Example prototype weights:

Component	Weight
Cash-flow stability	30%
Liquidity / expense coverage	25%
Revenue trend	20%
Transaction consistency	15%
Repayment behaviour	10%

10.3 Credit-readiness score - LOCKED

Component	Weight
Cash-flow stability	25%
Revenue consistency	20%
Growth trend	20%
Repayment behaviour	20%
Forecast liquidity	15%

Scoring disclaimer

These weights are prototype methodology for the hackathon. They are not Wema Bank underwriting policy and must not be presented as such.

11. Forecasting Specification

For a 12-hour build, correctness, stability and explainability are more important than model novelty.

11.1 Recommended model stack

1. Baseline: seasonal-naive or rolling-mean forecast.
2. Main candidate: scikit-learn HistGradientBoostingRegressor using engineered lag, rolling and calendar features.
3. Optional classical benchmark: statsmodels ExponentialSmoothing.
4. Choose the simplest model that performs acceptably on the synthetic holdout.

11.2 Features

- lag 1, 7, 14, 28 days
- rolling mean/std 7, 14, 30 days
- day-of-week, day-of-month, month
- end-of-month flag
- recent inflow/outflow trend
- recurring expense indicators

11.3 Evaluation

- Chronological split only; never random shuffle.
- Check for future-data leakage.
- Report MAE; use MAPE only when denominators are safe.
- Compare ML model to baseline.
- Use the baseline if the ML model is not clearly better.

12. Loan What-If Simulation

The simulator applies a proposed monthly repayment to the forecast period and recomputes projected liquidity. The calculation must be deterministic and backend-owned.

```
simulated_cashflow_t = baseline_forecast_net_cashflow_t - proposed_repayment_t
simulated_balance_t = prior_simulated_balance + simulated_cashflow_t
impact = simulated_balance_t - baseline_balance_t
```

Output labels:

- **MANAGEABLE**: no material deterioration in projected liquidity.
- **WATCH**: noticeable reduction in buffer; monitor.
- **HIGH_PRESSURE**: projected liquidity approaches or falls below the configured safety threshold.

13. API Contract - Freeze and Share

The API contract is the most important synchronization mechanism between the backend laptop and frontend laptop. The backend and frontend developers must not invent separate shapes.

Method	Endpoint	Purpose
GET	/api/v1/portfolio/summary	Portfolio headline metrics
GET	/api/v1/smes	Portfolio SME list
GET	/api/v1/smes/{id}	SME profile
GET	/api/v1/smes/{id}/health	Health metrics and insights
GET	/api/v1/smes/{id}/cashflow	Historical daily cash-flow series
GET	/api/v1/smes/{id}/forecast?horizon=90	Forecast points, status, drivers
GET	/api/v1/smes/{id}/credit-readiness	Score and components
GET	/api/v1/smes/{id}/accounts	Accounts + consent/source metadata
POST	/api/v1/smes/{id}/loan-simulation	Loan what-if simulation

GET	/healthz	Liveness
GET	/readyz	Readiness including database

13.1 API response rules

- Every response is JSON and uses snake_case keys on the backend.
- IDs are strings or integers consistently; choose one and freeze it in the schema package.
- Errors use HTTP status codes plus a stable {"detail": "..."} shape.
- No endpoint returns presentation-only strings instead of numeric data when the frontend needs to chart or calculate.
- Business logic stays in backend services, not route functions.

14. Repository Structure - Shared Source of Truth

```

repo/
├── README.md
├── .env.example
├── .gitignore
├── render.yaml
├── docs/
│   ├── PRD.md                # this document
│   ├── API_CONTRACT.md       # frozen request/response schemas
│   ├── DATA_DICTIONARY.md   # field meanings/enums/units
│   ├── TEST_PLAN.md          # test cases and results
│   ├── ARCHITECTURE.md        # system diagram + integration rules
│   ├── PROCESS_LOG.md         # agent/team checkpoints
│   └── DEMO_SCRIPT.md         # final judge path
├── backend/
│   ├── app/
│   │   ├── main.py
│   │   ├── core/
│   │   ├── api/
│   │   ├── models/
│   │   ├── schemas/
│   │   └── services/
│   │       ├── cashflow.py
│   │       ├── forecasting.py
│   │       ├── credit_score.py
│   │       ├── loan_simulator.py
│   │       └── data_providers/
│   │           ├── base.py
│   │           └── synthetic.py
│   ├── tests/
│   ├── scripts/seed_demo_data.py
│   └── requirements.txt
├── frontend/
│   ├── src/
│   │   ├── api/
│   │   ├── components/
│   │   ├── pages/
│   │   ├── hooks/
│   │   ├── types/
│   │   └── lib/
│   ├── tests/
│   └── package.json
└── data/
    └── synthetic/

```

Synchronization rule

If a teammate changes a shared contract, they must commit the contract change first, update dependent code, add a migration/test where needed, and log the change. Do not “temporarily” implement a different shape on another device.

15. Three-Person Ownership

Owner	Owns	Shared contracts they must respect
AI/ML Engineer	Synthetic data generation, analytics, forecasting, credit score, model evaluation	Transaction schema, ForecastPoint schema, CreditReadiness schema, API response shapes
Python Developer	FastAPI, Pydantic schemas, SQLAlchemy models, Alembic, PostgreSQL, adapters, backend tests, Render deploy	All API contracts; model/service interfaces
Fullstack Developer	React/TypeScript UI, components, charts, API hooks, UX, Vercel deploy	API response shapes; frontend types generated/maintained from contract

15.1 Parallel-work rule

- Every developer creates a feature branch from main at the start.
- Only small, focused commits; commit message starts with [ML], [BE], or [FE].
- Main branch is the integration branch.
- No force-pushing main.
- Before merge: run relevant tests and update PROCESS_LOG.md.
- When a branch is merged, teammates pull main before continuing.

16. Testing Strategy and Acceptance Gates

Test type	Required cases	Gate
Data validation	Schema, enums, duplicate IDs, missing values, balance consistency	Seed passes with zero invalid rows
Analytics unit	Inflow/outflow/net flow, growth, stability, liquidity	Known fixtures match expected ranges
Forecast	30/60/90 horizon, chronological split, no leakage, baseline comparison	Forecast service tests pass; baseline recorded
Credit readiness	0-100 bounds, weights sum to 100, expected directional behaviour	All scoring tests pass
Simulation	Repayment impact, edge amounts/tenors, no sign errors	Known scenario matches expected result
Backend API	200, 404, 422, health/readiness, response schema	P0 endpoints pass automated smoke
Frontend	Loading, error, empty, chart rendering, simulator state	Primary path passes component tests
Integration	Frontend -> Render API -> PostgreSQL	Full journey works from public URL
Security	Secrets, env vars, masked account IDs, CORS	No secrets in Git; CORS correct

16.1 Minimum automated tests

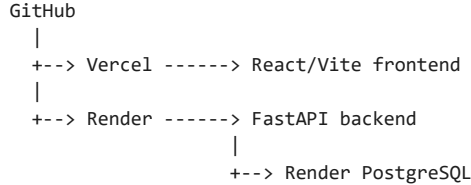
```
backend/tests/  
  test_health.py  
  test_data_validation.py  
  test_cashflow.py  
  test_forecast.py  
  test_credit_score.py  
  test_simulation.py  
  test_api_smoke.py
```

```
frontend/tests/  
  Portfolio.test.tsx  
  SMEDetail.test.tsx
```

Forecast.test.tsx
LoanSimulator.test.tsx

17. Deployment - Vercel + Render

17.1 Deployment topology



17.2 Required environment variables

Backend
DATABASE_URL=<Render PostgreSQL URL>
APP_ENV=production
CORS_ORIGINS=https://<vercel-domain>
SECRET_KEY=<generated secret>

Frontend
VITE_API_BASE_URL=https://<render-backend-domain>

17.3 Deployment acceptance

- GET /healthz returns success on the public Render URL.
- GET /readyz confirms database readiness.
- FastAPI Swagger /docs is reachable if not disabled for the demo.
- Frontend can load portfolio data from Render API.
- CORS permits only the required frontend origin(s).
- No secret is stored in GitHub.
- The public demo can complete the P0 journey without manual database edits.

18. Privacy, Security and Responsible AI

- Hackathon data is synthetic only.
- Account identifiers shown in the UI are masked.
- No bank passwords, API keys or credentials are stored in the repository.
- Customer consent is a first-class Open Banking concept: source, scope/status, timestamp and expiry are represented in the data model.
- External financial information is never treated as automatically visible to the bank; it is shown as customer-authorised in the prototype.
- Credit readiness and forecasts are decision-support outputs, not guaranteed outcomes.
- Any future production version would require formal privacy, security, model-risk, governance and regulatory review.

19. 12-Hour Build Schedule

Hour	ML/AI Engineer	Python Developer	Fullstack Developer	Gate
0-1	Agree fixtures, metrics and model interfaces	Repo, FastAPI, DB, schemas, API contract	Vite app, design system, API types	Contracts frozen
1-2.5	Synthetic generator +	Models, migrations, seed	Portfolio screen shell	Shared data loads

	health metrics	endpoint/script		
2.5-4	Forecast baseline + model	Health + forecast APIs	SME detail + history chart	Health + history works
4-5.5	Credit score + explanations	Credit API	Forecast screen	Forecast + score visible
5.5-6.5	Loan simulation formulas	Simulation endpoint + tests	Simulator UI	Headline feature works
6.5-7.5	Validate demo cases	Mock Open Banking adapter	Accounts/consent view + API integration	Unified data shown
7.5-9	Model QA + fixtures	Backend integration tests	UX polish + errors/loading	P0 end-to-end locally
9-10	Final model evaluation	Render deploy + DB	Vercel deploy	Public URLs work
10-11	Check all demo numbers	Fix integration defects	Final visual polish	Public smoke test passes
11-12	Rehearse model explanation	Rehearse technical Q&A	Rehearse click path	Demo locked

20. Cut List if Behind Schedule

1. Cut LLM narrative generation first.
2. Cut customer-facing consent screens; retain synthetic account/consent display.
3. Cut advanced model; ship the baseline if it is more reliable.
4. Cut Playwright if component/API tests and manual smoke are sufficient.
5. Cut secondary portfolio filters and cosmetic animations.
6. Never cut the core flow: Portfolio -> SME -> Health -> Forecast -> Credit Readiness -> Loan Simulation.

21. AI Agent Contract

Every AI coding agent receives the same instruction

Read the PRD, API contract, data dictionary, test plan and process log before coding. Do not redesign shared contracts. Implement only the assigned task and test it.

21.1 Mandatory agent instructions

- Inspect repository before edits.
- List files to change before implementation.
- Use existing package choices and folder structure.
- Never invent frontend mock logic that conflicts with backend calculations.
- Never change score weights or formulas silently.
- Never use real banking credentials or data.
- Run tests before marking DONE.
- Append a checkpoint to docs/PROCESS_LOG.md.
- If blocked, log the blocker rather than silently changing the architecture.

21.2 Master OpenCode prompt

You are a coding agent working on a 12-hour banking hackathon project. The repository and PRD are the source of truth.

Read first:

- docs/PRD.md
- docs/API_CONTRACT.md
- docs/DATA_DICTIONARY.md
- docs/TEST_PLAN.md
- docs/ARCHITECTURE.md

- docs/PROCESS_LOG.md

Stack is locked:

React + TypeScript + Vite; Tailwind; Recharts; TanStack Query.

Python + FastAPI + Pydantic v2 + SQLAlchemy + Alembic.

PostgreSQL. pandas + NumPy + scikit-learn; optional statsmodels baseline.

Vercel frontend; Render backend + PostgreSQL.

Rules:

1. P0 beats P1/P2.
2. Do not redesign shared contracts.
3. Backend owns business calculations.
4. Frontend consumes API data; it must not duplicate business formulas.
5. Use synthetic data only.
6. Open Banking is consent-based; the prototype only simulates external accounts.
7. Credit readiness is prototype decision support, not a lending approval engine.
8. Time-series evaluation must be chronological; check leakage.
9. Add tests for every changed behaviour.
10. Log every meaningful checkpoint.

TASK:

[INSERT ASSIGNED TASK HERE]

Before coding:

- inspect the repository;
- identify relevant files;
- identify contract dependencies.

After coding:

- run relevant tests;
- report exact files changed;
- update docs/PROCESS_LOG.md with:
[timestamp] [role] [task-id]

Status:

Goal:

Changes:

Files changed:

Tests run:

Test result:

Decisions:

Blockers:

Next checkpoint:

Time remaining:

Do not mark the task DONE unless implementation, tests, and process log are complete.

22. Change Control and Conflict Resolution

Because the team is working on different devices, uncontrolled changes are a bigger risk than coding speed. Use this process:

1. Any developer who discovers a missing requirement first records it in docs/PROCESS_LOG.md.
2. If the change affects only their local implementation and not a shared contract, they may proceed.
3. If it changes an API, schema, score formula, model input, deployment contract or user journey, stop and log a change request.
4. The team lead or agreed decision-maker approves the change in the log.
5. Update the PRD/API/data docs, then implement the dependent changes together.
6. Re-run all affected tests and the end-to-end smoke test.

22.1 Change request template

CR-XXX

Requested by:

Date/time:

Reason:
Current contract:
Proposed change:
Why P0/P1 value justifies it:
Affected files/teams:
Tests required:
Decision: APPROVED / REJECTED
Decision maker:

23. Final Definition of Done

Build is DONE only when the same product works from a clean checkout

A new teammate can clone the repository, install dependencies, seed the same deterministic data, run the tests, start frontend/backend, and see the same P0 calculations and UI flow without relying on hidden files or verbal instructions.

1. API and data contracts are frozen and committed.
2. Synthetic dataset is deterministic and seeded.
3. Portfolio -> SME -> Health -> Forecast -> Credit -> Loan Simulation works locally.
4. Open Banking view is clearly marked as consented/synthetic.
5. Automated P0 backend tests pass.
6. Critical frontend tests pass.
7. Public Vercel + Render deployment works.
8. No secrets or real customer data are committed.
9. Process log records final checkpoints.
10. The team can run the complete judge demo without manual code changes.

24. Submission Checklist

- Public frontend URL tested.
- Public backend health endpoint tested.
- GitHub repository clean and documented.
- README contains setup + deployment + demo credentials/steps where applicable.
- Synthetic fixture IDs are stable.
- All demo numbers match the final seeded database.
- One person drives the demo; the other two handle technical questions.
- No unsupported claim of live Wema or Open Banking integration.
- No claim that the credit score is Wema policy.
- Demo is reproducible from the shared repository.

25. Cross-Device Team Synchronization Pack

This section is mandatory for all three teammates and every AI coding agent. It exists to prevent different devices or agents from implementing different interpretations of the product.

25.1 Source-of-truth hierarchy

1. REQUIREMENTS.md — what problem/user outcome the system must satisfy.
2. PRD.md — what we are building and what is explicitly out of scope.
3. API_CONTRACT.md — exact backend/frontend request and response shapes.
4. DATA_DICTIONARY.md — exact field names, enums, units, date conventions and meanings.
5. ARCHITECTURE.md — system boundaries and integration rules.
6. TEST_PLAN.md — acceptance criteria and verification expectations.
7. PROCESS_LOG.md — live record of decisions, progress, blockers and approved changes.
8. Code — implementation of the above contracts; code must not silently redefine them.

25.2 Mandatory shared artifacts

docs/REQUIREMENTS.md

docs/PRD.md

docs/API_CONTRACT.md

docs/DATA_DICTIONARY.md

docs/DB_SCHEMA.md

docs/ARCHITECTURE.md

docs/ML_CONTRACT.md

docs/TEST_PLAN.md

docs/PROCESS_LOG.md

docs/DECISIONS.md

AGENTS.md at repository root

25.3 Requirements-gathering contract

Before coding, the team must capture and freeze the following product requirements in docs/REQUIREMENTS.md:

Primary user: Wema SME banking / credit / relationship-management teams.

Primary job: understand SME financial health and future liquidity well enough to support better-informed lending decisions.

Secondary job: give the bank a broader view when SMEs use multiple financial institutions, subject to customer-authorised Open Banking access.

Primary success metric for the hackathon: a judge can select a synthetic SME and complete Portfolio → Health → Forecast → Credit Readiness → Loan Simulation without manual code/database changes.

The system must remain useful using only Wema-native transaction data; external-account data is an enhancement, not a hard dependency.

All credit outputs are prototypes and decision support only.

All Open Banking data in the hackathon is synthetic and clearly marked as consented/connected demo data.

Every feature must map to a requirement ID such as FR-001, NFR-001 or INT-001. No feature is considered complete without an acceptance criterion.

25.4 Exact process log format

PROCESS_LOG.md

Team Clock

- Start: _____
- Current hour: _____
- Deadline: _____
- Hours remaining: _____

Checkpoints

- ☐ Repo + API/data contract frozen
- ☐ Synthetic data seeded
- ☐ Health metrics working
- ☐ Forecast working
- ☐ Credit readiness working
- ☐ Loan simulator working
- ☐ Frontend connected
- ☐ Backend deployed
- ☐ Frontend deployed
- ☐ End-to-end smoke test passed
- ☐ Demo rehearsed

Agent / Team Updates

[YYYY-MM-DD HH:MM] [ROLE] [TASK-ID]

Status: DONE | PARTIAL | BLOCKED

Goal:

Changes:

Files changed:

Tests run:

Test result:

Decisions:

Blockers:

Next checkpoint:

Time remaining:

Next update

Status:

Goal:

Changes:

Files changed:

Tests run:

Test result:

Decisions:

Blockers:

Next checkpoint:

Time remaining:

This exact template is also included as docs/PROCESS_LOG.md in the companion build pack.

25.5 Shared schemas to freeze

DB schema: SME, Account, Transaction, Loan, ForecastPoint and optional Consent entities.

API schema: endpoint paths, methods, query params, request bodies, response bodies, status codes and error format.

Data dictionary: field names, types, enum values, units, nullability and examples.

ML contract: exact input columns, preprocessing, output fields, score ranges, evaluation metrics and model version.

Frontend contract: exact TypeScript representations of API responses; frontend must not reproduce backend formulas.

State/enums: health_status = healthy|watch|risk; forecast_status = healthy|watch|high_pressure; credit_readiness = 0–100; account_source = wema|open_banking_demo; consent_status = active|expired|revoked|not_connected.

Money convention: for hackathon API examples, monetary values are Nigerian Naira and numeric; if the team changes to minor-unit integers or Decimal strings, that decision must be frozen in DATA_DICTIONARY.md before parallel coding.

Date convention: database UTC; UI converts to Africa/Lagos; API timestamps use ISO 8601.

25.6 Contract-change rule

If a change affects an API field, schema, formula, ML feature, scoring weight, enum, endpoint, deployment variable, or demo flow, the developer/agent must stop, create a change request in docs/DECISIONS.md, obtain team approval, update the affected contract document, then update all dependent code together.

25.7 Master OpenCode prompt — exact team version

OpenCode Master Prompt — SME Financial Intelligence Hackathon

You are an engineering agent working inside a ****12-hour hackathon build****. You must follow the project PRD and existing repository contracts exactly.

Project goal

Build an AI-powered SME financial intelligence platform for a banking use case. The platform analyses ****synthetic SME transaction data****, optionally combines ****customer-authorised external-account data through a mock Open Banking adapter****, and produces:

- financial health metrics
- historical cash-flow intelligence
- 30/60/90-day cash-flow forecasts
- explainable prototype credit readiness
- loan what-if simulation

The hackathon prototype must not claim live Wema or third-party-bank integration unless a real authorised sandbox/API is actually connected.

Non-negotiable product rules

1. Synthetic data only for the hackathon.
2. Open Banking is consent-based. Never design the system as if Wema can see another customer's bank account without authorisation.
3. Credit readiness is decision support, not Wema's actual underwriting model. Never output approval/decline.
4. P0 features beat P1/P2 features.
5. Prefer simple, deterministic, explainable methods over complex models.
6. Do not introduce new frameworks or large dependencies without a concrete blocker.
7. Preserve the frozen API/data contracts unless you identify a genuine defect.
8. Never commit secrets, real credentials, or real banking data.
9. Do not overwrite unrelated teammate work.
10. Every meaningful checkpoint must be written to `docs/PROCESS\_LOG.md`.

Locked stack

Frontend: React + TypeScript + Vite + Tailwind CSS + Recharts + TanStack Query.

Backend: Python + FastAPI + Pydantic v2 + SQLAlchemy 2.x + Alembic.

Database: PostgreSQL.

ML: pandas + NumPy + scikit-learn; optional statsmodels baseline.

Testing: pytest + pytest-asyncio + httpx; Vitest + React Testing Library; Playwright only if time remains.

Deployment: Vercel frontend; Render FastAPI backend + Render PostgreSQL.

Read before coding

- docs/REQUIREMENTS.md
- docs/PRD.md
- docs/API_CONTRACT.md
- docs/DATA_DICTIONARY.md
- docs/DB_SCHEMA.md
- docs/ML_CONTRACT.md
- docs/ARCHITECTURE.md
- docs/TEST_PLAN.md
- docs/PROCESS_LOG.md
- docs/DECISIONS.md
- AGENTS.md

TASK

Replace this with the specific task assigned to you.

Execution protocol

1. Inspect the repository and relevant contracts before editing.
2. State the files and contracts you expect to touch.
3. Implement the smallest solution that satisfies the requirement.
4. Add/update tests for changed behaviour.
5. Run tests immediately.
6. Append the required checkpoint to docs/PROCESS_LOG.md.
7. If blocked, log the blocker; do not silently invent a different architecture.
8. Return a handoff containing changes, files, tests/results, blockers and exact next step.

ML rules

- Chronological train/validation only.
- Explicitly check future-data leakage.
- Compare candidate model against a simple baseline.
- Prefer MAE when MAPE is unstable.
- Keep credit scoring weights explicit and testable.

Forecast recommendation

Use daily net cash flow per SME.

Features: lag 1, 7, 14, 28; rolling mean/std 7, 14, 30; day-of-week; day-of-month; month; end-of-month flag; recent inflow/outflow trend; recurring expense indicators.

Candidate model: HistGradientBoostingRegressor.

Baseline: seasonal-naive or rolling mean.

Use the simpler model in the demo if it is as good or better.

Credit-readiness prototype weights

- cash-flow stability: 25%
- revenue consistency: 20%
- growth trend: 20%
- repayment behaviour: 20%
- forecast liquidity: 15%

Score range must be 0–100.

Loan simulation

The simulator is decision support only. Larger projected repayments must not improve projected liquidity under otherwise identical conditions.

API contract — MUST PRESERVE

- GET /api/v1/portfolio/summary
- GET /api/v1/smes
- GET /api/v1/smes/{id}
- GET /api/v1/smes/{id}/health
- GET /api/v1/smes/{id}/cashflow
- GET /api/v1/smes/{id}/forecast?horizon=90
- GET /api/v1/smes/{id}/credit-readiness
- GET /api/v1/smes/{id}/accounts
- POST /api/v1/smes/{id}/loan-simulation
- GET /healthz
- GET /readyz

Process log — mandatory exact format

Append after every meaningful checkpoint:

[YYYY-MM-DD HH:MM] [ROLE] [TASK-ID]

Status: DONE | PARTIAL | BLOCKED

Goal:

Changes:

Files changed:

Tests run:

Test result:

Decisions:

Blockers:

Next checkpoint:

Time remaining:

Also record deviations, dependency changes, API/schema changes and intentionally deferred bugs.

Definition of done

Do not mark DONE unless implementation exists, relevant tests pass, no unrelated contract is broken, process log is updated, and another teammate can continue without verbal context.

26. Recommended additional contract files

These are the files the team should create at the start of Hour 1. They are intentionally small and concrete.

REQUIREMENTS.md — requirement IDs, user stories, assumptions, acceptance criteria, non-functional requirements, dependencies.

API_CONTRACT.md — exact endpoints + JSON request/response examples + errors.

DATA_DICTIONARY.md — every data field, enum, type, unit and date convention.

DB_SCHEMA.md — tables, columns, keys, indexes and relationships.

ML_CONTRACT.md — model inputs, feature generation, outputs, score formulas, evaluation and version.

ARCHITECTURE.md — component diagram, ownership boundaries, adapter rules.

TEST_PLAN.md — unit/integration/E2E cases and acceptance gates.

PROCESS_LOG.md — live team/agent progress.

DECISIONS.md — approved changes and rationale.

AGENTS.md — root-level instructions for every AI coding agent.

These documents, not individual laptop assumptions, are the shared source of truth. Commit them before parallel coding begins.